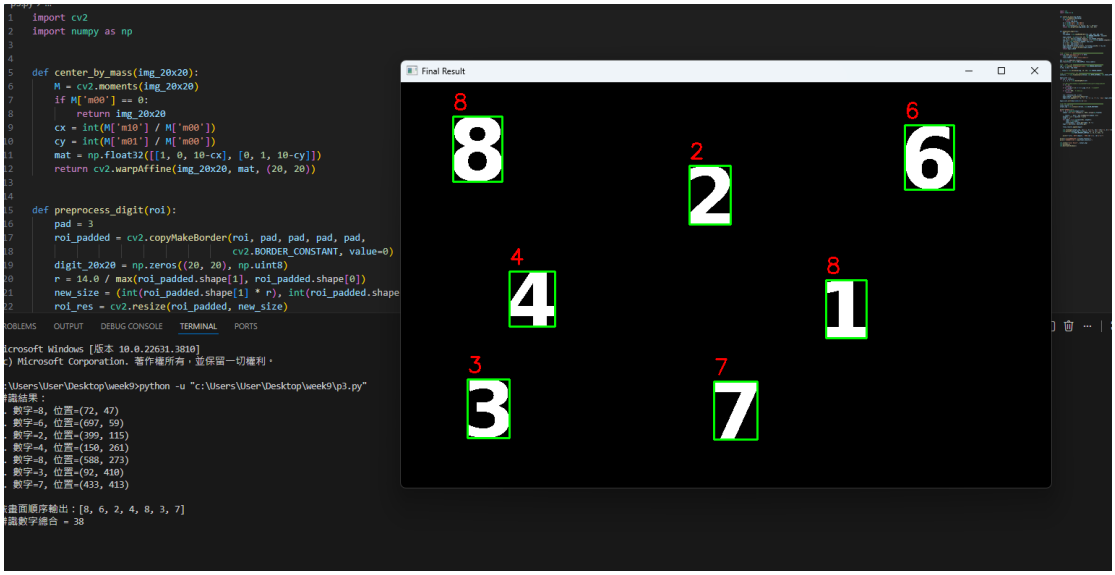


```python
import cv2
import numpy as np


def center_by_mass(img_20x20):
    M = cv2.moments(img_20x20)
    if M['m00'] == 0:
        return img_20x20
    cx = int(M['m10'] / M['m00'])
    cy = int(M['m01'] / M['m00'])
    mat = np.float32([[1, 0, 10-cx], [0, 1, 10-cy]])
    return cv2.warpAffine(img_20x20, mat, (20, 20))


def preprocess_digit(roi):
    pad = 3
    roi_padded = cv2.copyMakeBorder(roi, pad, pad, pad, pad,
                                    cv2.BORDER_CONSTANT, value=0)
    digit_20x20 = np.zeros((20, 20), np.uint8)
    r = 14.0 / max(roi_padded.shape[1], roi_padded.shape[0])
    new_size = (int(roi_padded.shape[1] * r), int(roi_padded.shape[0] *
r))
    roi_res = cv2.resize(roi_padded, new_size)
    tx = (20 - new_size[0]) // 2
    ty = (20 - new_size[1]) // 2
    digit_20x20[ty:ty+new_size[1], tx:tx+new_size[0]] = roi_res
```

```
    digit_20x20 = center_by_mass(digit_20x20)
    return digit_20x20


# ── 載入訓練資料 ──────────────────────────────────────
with np.load('knn_digit.npz') as data:
    train = data['train']
    train_labels = data['train_labels']

knn = cv2.ml.KNearest_create()
knn.train(train, cv2.ml.ROW_SAMPLE, train_labels)

# ── 讀圖與二值化 ──────────────────────────────────────
img = cv2.imread('hiddendigits.png', cv2.IMREAD_GRAYSCALE)
h_img, w_img = img.shape

_, thresh = cv2.threshold(img, 10, 255, cv2.THRESH_BINARY)

# ── 找輪廓、擷取數字 ROI ──────────────────────────────
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

digit_list = []
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)

    # ★ 過濾條件：排除太小、太大（外框輪廓）、長寬比異常的輪廓
    if h < 15:
        continue
    if w > w_img * 0.8 or h > h_img * 0.8:  # 排除外框
        continue
    if w * h < 500:  # 排除雜訊
        continue

    roi = thresh[y:y+h, x:x+w]
    digit_20x20 = preprocess_digit(roi)
    digit_list.append({'y': y, 'x': x, 'w': w, 'h': h, 'img':
digit_20x20})
```

```python
digit_list.sort(key=lambda d: d['y'])

# ── 辨識 ──────────────────────────────────────────────────
final_results = []
output_img = cv2.cvtColor(thresh, cv2.COLOR_GRAY2BGR)

print("辨識結果：")
for i, d in enumerate(digit_list):
    sample = d['img'].reshape((1, 400)).astype(np.float32)

    _, result, _, dist = knn.findNearest(sample, k=1)
    weights = 1.0 / (dist[0] + 1e-5)
    votes = {}
    for label, w in zip(result[0], weights):
        label = int(label)
        votes[label] = votes.get(label, 0) + w
    digit = max(votes, key=votes.get)

    final_results.append(digit)

    cv2.rectangle(output_img, (d['x'], d['y']), (d['x']+d['w'],
d['y']+d['h']), (0, 255, 0), 2)
    cv2.putText(output_img, str(digit), (d['x'], d['y']-10),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    print(f"{i+1}. 數字={digit}, 位置=({d['x']}, {d['y']})")

print(f"\n依畫面順序輸出：{final_results}")
print(f"辨識數字總合 = {sum(final_results)}")

cv2.imshow('Final Result', output_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```